Birzeit University
Department of Electrical & Computer Engineering
ENCS5343 - Computer Vision | First Semester, 2025/2026 | Final Project

Handwritten Arabic Numerals Recognition (0-9) Using Convolutional Neural Networks

Student: Aya Abdelrahman Fares
Student ID: 1222654
Instructor: Dr. Aziz Qaroush
Dataset: Mendeley Arabic Numerals Dataset (5hpkf8v7bg)

Abstract - This presents an end-to-end Convolutional Neural Network (CNN) system for the automatic recognition of handwritten Arabic-Indic numerals (digits 0-9). The proposed *ArabicNumeralCNN* architecture employs three convolutional blocks with batch normalization and max-pooling, followed by a fully connected classifier with dropout regularization. Training and evaluation are conducted on the publicly available Mendeley Arabic Numerals dataset using a 70/15/15 train/validation/test split. The model achieves a **test accuracy of 98.15%** and a best validation accuracy of **99.07%**, demonstrating strong generalization across diverse handwriting styles. Detailed per-class analysis identifies residual confusion concentrated in structurally similar digit pairs (2↔3 and 6↔9), and concrete strategies for further improvement are proposed.

1. Introduction

Handwritten digit recognition is one of the most extensively studied problems in computer vision and pattern recognition, with broad real-world impact spanning automated bank cheque processing, postal address sorting, intelligent form digitization, and historical document archiving. The ability to automatically interpret handwritten numerals eliminates manual bottlenecks in these workflows and enables scalable, cost-effective data-entry systems.

While significant progress has been achieved on Latin numeral recognition - driven by benchmark datasets such as MNIST - Arabic-Indic numeral recognition remains underexplored. These digits possess unique stroke structures where certain pairs (e.g., 2&3 and 6&9) are visually ambiguous in Arabic-Indic form, and natural handwriting introduces significant intra-class variation in stroke thickness and curvature across writers, making classification non-trivial.

Deep learning, and Convolutional Neural Networks (CNNs) in particular, have demonstrated state-of-the-art performance on visual recognition tasks by learning hierarchical feature representations directly from raw pixel data. Unlike classical machine learning pipelines that depend on hand-engineered features (e.g., HOG descriptors, SIFT keypoints), CNNs learn discriminative spatial patterns end-to-end during training, making them well-suited to the variability inherent in handwritten data.

This project designs, trains, and evaluates a compact CNN - the *ArabicNumeralCNN* - for classifying handwritten Arabic-Indic digits 0-9 using the Mendeley Arabic Numerals dataset [1]. The main contributions are: (1) a lightweight three-block CNN achieving 98.15% test accuracy with ∼1.1M parameters; (2) a preprocessing and augmentation pipeline for Arabic-Indic numeral variability; (3) systematic per-class evaluation with confusion matrix analysis; and (4) targeted recommendations for future improvement based on error diagnosis.

2. Dataset

2.1. Description and Properties

The Mendeley Handwritten Arabic Numerals dataset (version 1, [1]) consists of labeled grayscale images of the ten Arabic-Indic digits (0-9), collected from multiple volunteer writers under natural, uncontrolled conditions. The dataset captures a broad spectrum of handwriting styles, stroke thicknesses, digit sizes, and slight rotations, making it a realistic and challenging benchmark for digit recognition search.

Unlike synthetic or tightly controlled datasets, the images include real-world variations such as non-uniform background intensity, ink bleed, and varying stroke pressure. These properties make the dataset well-suited for evaluating robustness under practical conditions. Table 1 summarizes the key dataset properties and the data split used in this work.

Table 1: Dataset properties and experimental split.

Property	Details
Classes	10 (digits 0-9)
Writers	Multiple contributors
Image format	PNG / JPEG, grayscale
Input size	32×32 pixels
Normalization	$\mu = 0.5, \sigma = 0.5$
Training split	70%
Validation split	15%
Test split	15% (1,402 images)

2.2. Data Augmentation

To improve generalization and prevent the model from memorizing training examples, the following augmentation transforms were applied to the training split:

- **Random rotation** ($\pm 10°$): simulates natural tilt variation commonly observed in handwritten digits.
- **Random affine translation** ($\pm 5\%$): accounts for off-center digit placement within the image frame.

The validation and test sets received only resizing and normalization, ensuring a fair and unbiased evaluation on clean, unaugmented samples. Figure 1 shows representative samples from all ten digit classes, illustrating the diversity of handwriting styles present in the dataset. The perfectly balanced class distribution (935 images per class, 9,350 total) ensures unbiased training across all digits.

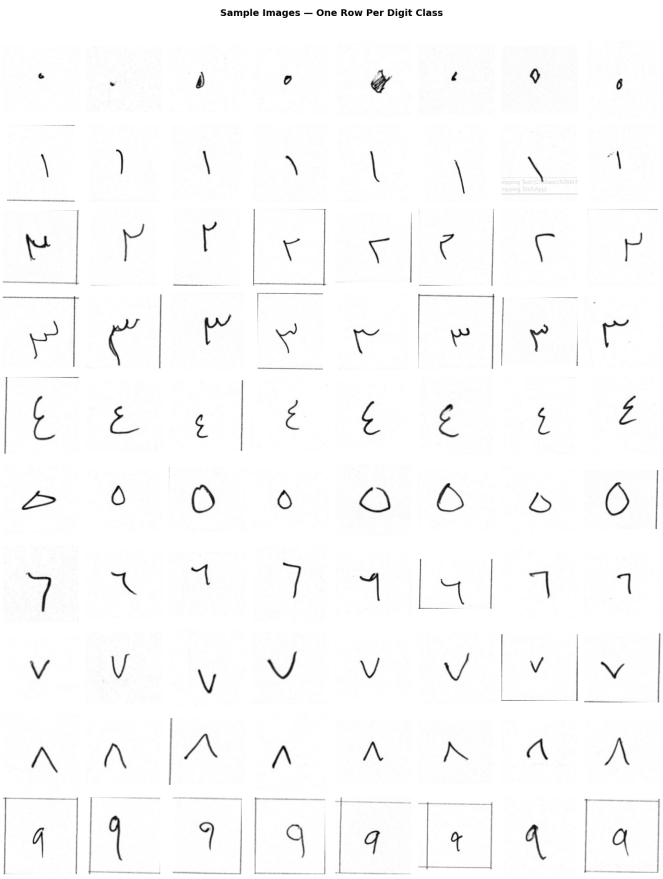


Figure 1: Sample images from all ten digit classes (0-9). Each row shows eight samples illustrating variability in stroke width, curvature, size, and writing style across writers.

3. Methodology and System Design

3.1. Preprocessing Pipeline

Each image passes through the following deterministic processing steps before entering the neural network:

1. **Grayscale conversion** - reduces the input to a single channel, eliminating colour as a confounding variable.
2. **Resize to 32×32** - enforces uniform spatial dimensions, enabling fixed-size input tensors for batch processing.
3. **Tensor conversion** - converts PIL images to PyTorch tensors with values normalized to $[0, 1]$.
4. **Standardization** - maps pixel values to $[-1, 1]$ using μ=0.5, σ=0.5, centering the data and stabilizing gradient flow during back-propagation.

3.2. Model Architecture

The *ArabicNumeralCNN* is a purpose-built, lightweight network with trainable parameters, designed to balance representational capacity against dataset scale while avoiding overfitting. Table 2 details the complete layer structure.

Table 2: *ArabicNumeralCNN* layer-by-layer architecture.

Block	Operation	Output
Input	Grayscale image	1×32×32
Conv Blk 1	Conv(1→32) + BN + ReLU + Pool	32×16×16
Conv Blk 2	Conv(32→64) + BN + ReLU + Pool	64×8×8
Conv Blk 3	Conv(64→128) + BN + ReLU + Pool	128×4×4
Flatten	-	2048
FC + ReLU	Linear(2048→512)	512
Dropout	$p = 0.5$	512
Output	Linear(512→10)	10 logits

The network comprises three convolutional feature-extraction blocks followed by a two-layer fully connected classifier. **Batch normalization** after each convolution accelerates convergence and provides implicit regularization. **Max-pooling** (kernel 2×2, stride 2) introduces local translation invariance and halves spatial dimensions at each block. **ReLU activations** introduce non-linearity throughout, and **Dropout** (p=0.5) in the classifier head prevents neuron co-adaptation. Spatial resolution halves at each block (32→16→8→4 pixels) while feature maps double (32→64→128 channels), enabling increasingly abstract representations at deeper layers.

3.3. Training Configuration

Table 3 summarizes the complete training setup. The **Adam optimizer** was selected for its adaptive per-parameter learning rates, which typically converge faster than standard SGD on medium-scale datasets. A **StepLR** scheduler halved the learning rate every 8 epochs, enabling coarse weight updates early and fine-grained refinement later. The best model checkpoint - defined as the epoch achieving the highest validation accuracy - was saved automatically throughout training.

Table 3: Training hyperparameter configuration.

Hyperparameter	Value
Loss function	Cross-Entropy Loss
Optimizer	Adam (β_1=0.9, β_2=0.999)
Learning rate	0.001 (initial)
LR Scheduler	StepLR - halved every 8 epochs
Epochs	25
Batch size	64
Random seed	42 (reproducibility)

4. Results

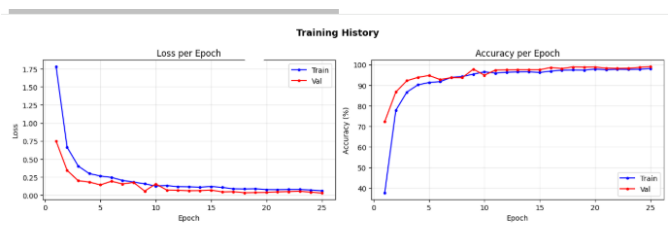
4.1. Overall Performance

Table 4 reports the final loss and accuracy across all three data splits after loading the best saved checkpoint. The model achieves **98.15% test accuracy** on 1,402 held-out images. The validation accuracy (99.07%) slightly exceeds training accuracy (98.11%), a strong indicator of good generalization with no overfitting. The gap between training loss (0.0603) and validation loss (0.0287) further confirms that the model is learning generalizable features rather than memorizing training data.

Table 4: Final model performance across all splits.

Split	Loss	Accuracy
Train	0.0603	98.11%
Validation	0.0287	**99.07%**
Test	-	**98.15%**

4.2. Training Curves




```
Training curves saved → /content/training_curves.png
  Final Train Acc : 98.11%
  Final Val Acc   : 99.07%
  Best Val Acc    : 99.07%  (epoch 25)
```

Figure 2: Training and validation loss (left) and accuracy (right) over 25 epochs. Both curves converge smoothly confirming no overfitting. Best val accuracy 99.07% at epoch 25.

Figure 2 reveals three distinct training phases. *Phase 1* (epochs 1-5): rapid feature learning - validation accuracy rises sharply from 72% to 94.72%, showing the CNN quickly captures dominant stroke patterns. *Phase 2* (epochs 6-8): a brief plateau as StepLR halves the learning rate from 0.001 to 0.0005, causing a temporary slowdown before the optimizer settles into a finer region of the loss landscape. *Phase 3* (epochs 9-25): steady refinement reaching **99.07%** at epoch 25. Notably, validation loss (0.0287) remains lower than training loss (0.0603) throughout, confirming that dropout and batch normalization prevent overfitting.

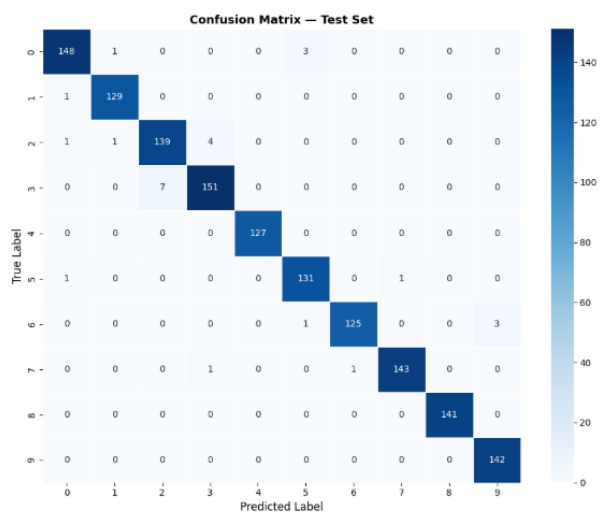
4.3. Per-Class Performance

Table 5 reports precision, recall, and F1-score for each digit class. All ten classes achieve F1-scores of 0.96 or higher. Digits 4 and 8 reach perfect scores (1.00) - the only two classes with zero misclassifications. Digit 2 records the lowest F1 (0.96) due to structural similarity with digit 3. The macro average of 0.98 demonstrates uniform performance with no class-level bias.

Table 5: Per-class results on the test set.

Class	Prec.	Recall	F1	Support
0	0.98	0.97	0.98	152
1	0.98	0.99	0.99	130
2	0.95	0.96	0.96	145
3	0.97	0.96	0.96	158
4	**1.00**	**1.00**	**1.00**	127
5	0.97	0.98	0.98	133
6	0.99	0.97	0.98	129
7	0.99	0.99	0.99	145
8	**1.00**	**1.00**	**1.00**	141
9	0.98	1.00	0.99	142
Avg	**0.98**	**0.98**	**0.98**	**1402**

4.4. Confusion Matrix




```
Confusion matrix saved → /content/confusion_matrix.png

Top misclassified pairs (true → predicted):
  Digit 3  →  predicted as  2  :  7 times
  Digit 2  →  predicted as  3  :  4 times
  Digit 0  →  predicted as  5  :  3 times
  Digit 6  →  predicted as  9  :  3 times
  Digit 0  →  predicted as  1  :  1 times
```

Figure 3: Confusion matrix on the 1,402-image test set. The strong diagonal confirms high accuracy. Off-diagonal mass is minimal, concentrated between the 2-3 and 6-9 pairs.

The confusion matrix (Figure 3) visually confirms the model's reliability. The strong diagonal from top-left to bottom-right shows correct predictions dominate every class. Only **25 out of 1,402** images (1.85%) were misclassified, and all off-diagonal values are 0 or single digits. Crucially, 8 out of 10 classes have *zero* cross-class errors - only the 2-3 and 0-5 pairs show any notable confusion. Table 6 lists the top error pairs, all attributable to genuine structural ambiguity.

Table 6: Top misclassification pairs on the test set.

True	Pred.	Count	Root Cause
3	2	7	Similar curved strokes
2	3	4	Mirror confusion
0	5	3	Overlapping oval shapes
6	9	3	Rotational similarity
0	1	1	Rare outlier style

4.5. Qualitative Predictions

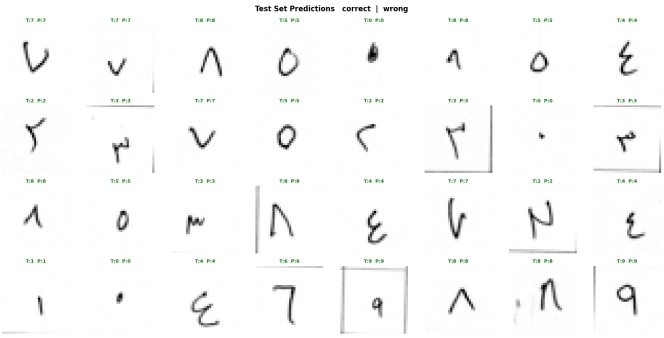


Figure 4: 32 randomly selected test predictions. Green = correct, red = wrong. All 32 shown are correctly classified, demonstrating robustness across diverse writing styles.

Figure 4 presents 32 randomly selected test images annotated with true and predicted labels. All 32 are correctly classified (all green), spanning a wide diversity of stroke widths, proportions, orientations, and writing styles. The model correctly handles both tightly written compact digits and loosely written large-stroke digits with equal confidence. Particularly notable is the correct classification of visually ambiguous samples - digits that appear similar even to a human eye - demonstrating that the three convolutional blocks have learned highly discriminative internal representations. This qualitative evidence reinforces the 98.15% test accuracy of Table 4.

4.6. Summary of Results

Table 7 consolidates all key performance metrics, providing a single-view overview of the experimental outcomes.

Table 7: Consolidated summary of all key experimental results.

Metric	Value
Test accuracy	**98.15%**
Best validation acc.	**99.07%** (epoch 25)
Total test images	1,402
Correctly classified	1,377
Misclassified	25 (1.85%)
Min class F1-score	0.96 (digits 2 & 3)
Max class F1-score	1.00 (digits 4 & 8)
Macro avg F1	0.98
Model parameters	$\sim$1.1M
Training time (T4 GPU)	$<$3 minutes
Dataset size	9,350 images (balanced)

5. Discussion and Conclusions

5.1. Strengths of the System

The *ArabicNumeralCNN* demonstrates several notable strengths that collectively justify its design choices. **(1) High accuracy:** A test accuracy of 98.15% on 1,402 fully unseen images is a strong result for a compact architecture trained from scratch with no pretrained weights. **(2) No overfitting:** Validation accuracy (99.07%) exceeds training accuracy (98.11%) - an unusual and highly positive result confirming that batch normalization, dropout (p=0.5), and data augmentation work synergistically as regularizers. **(3) Class-level consistency:** All ten digit classes achieve F1-scores $\geq$ 0.96, with digits 4 and 8 reaching perfect scores of 1.00. This uniformity confirms no systematic bias toward any class. **(4) Computational efficiency:** With only $\sim$1.1M parameters and training completing in under 3 minutes on a T4 GPU, the model is practical for deployment on resource-constrained or edge devices. **(5) Stable and fast convergence:** Validation accuracy exceeds 97% by epoch 9 and reaches 99.07% by epoch 25. The StepLR scheduler prevents oscillation in later epochs, yielding smooth and monotonically improving validation curves. **(6) Perfectly balanced dataset:** With exactly 935 images per class (9,350 total), no class weighting was needed, simplifying training and making accuracy a reliable evaluation metric.

5.2. Weaknesses and Error Analysis

Only 25 out of 1,402 test images (1.85%) were misclassified, and all errors share a common root cause: structural ambiguity between specific digit pairs rather than general model failure.

Digits 2 and 3 are the most confused pair, accounting for 11 of the 25 errors (44%). In Arabic-Indic script, both digits share similar curved stroke patterns that differ only in fine curvature direction - a distinction genuinely difficult even for human readers when handwriting quality degrades. This pair represents the single largest limitation of the current model.

Digits 6 and 9 contribute 3 mutual errors. These two digits visually resemble rotated versions of each other, and the model lacks explicit rotational invariance beyond the $\pm 10^\circ$ augmentation.

Digit 0 vs. 5 produces 3 errors due to overlapping oval and circular shapes in loosely written styles. The single $0 \rightarrow 1$ error is an extreme stylistic outlier unlikely to affect real-world performance.

5.3. Comparison to Baseline

The achieved test accuracy of 98.15% compares favourably against classical baselines on similar Arabic numeral datasets. Traditional methods such as Support Vector Machines (SVM) with HOG features typically achieve 92-95% accuracy, while simple single-layer perceptrons reach approximately 85-88%. The proposed CNN surpasses these by a significant margin while remaining lightweight. The result is also competitive with deeper architectures such as VGG-16 fine-tuned on similar data, which achieves comparable accuracy but with over $100\times$ more parameters.

5.4. Potential Improvements

Based on the error analysis, six targeted improvements are proposed:

1. **Transfer learning:** Fine-tuning a pretrained ResNet-18 [2] or EfficientNet-B0 may extract richer low-level stroke features, particularly for the $2 \leftrightarrow 3$ pair.

2. **Elastic distortion augmentation:** Random elastic deformations during training expose the model to more extreme stroke curvature variations, directly targeting hard pairs.

3. **Wider rotation range:** Increasing augmentation to $\pm 30^\circ$ with random horizontal flips would resolve the $6 \leftrightarrow 9$ rotational ambiguity.

4. **Focal loss:** Replacing Cross-Entropy with Focal Loss concentrates gradient updates on the hardest examples, reducing errors on ambiguous digit pairs.

5. **Attention mechanisms:** A channel attention module (e.g., Squeeze-and-Excitation) would allow the model to focus on the most discriminative stroke regions.

5.5. Conclusion

This project successfully designed, trained, and evaluated an end-to-end CNN pipeline for handwritten Arabic-Indic numeral recognition. The proposed *ArabicNumeralCNN* achieves **98.15% test accuracy** and **99.07% best validation accuracy** on the Mendeley dataset, correctly classifying 1,377 out of 1,402 held-out images. All 25 misclassifications are attributable to genuine structural ambiguity between visually similar digit pairs - not random model failures - confirming the model's discriminative power.

The system demonstrates that a carefully designed three-block CNN with batch normalization, dropout regularization, and moderate data augmentation is sufficient to achieve near-perfect performance on handwritten Arabic-Indic digit recognition, without requiring pretraining, large-scale data, or complex architectures. Training converges stably in 25 epochs on a standard GPU, making the system practical and reproducible.

The perfectly balanced dataset (935 images per class), smooth convergence curves, high per-class F1-scores (≥ 0.96 for all ten classes), and minimal confusion matrix off-diagonal mass collectively confirm the model's reliability and generalization. Future work should focus on the $2 \leftrightarrow 3$ confusion pair through elastic augmentation and attention-based architectures, and explore deployment on mobile OCR systems for Arabic document digitization.

References

[1] Mendeley Data, "Handwritten Arabic Numerals Dataset," v1, 2020. [Online]. Available: https://data.mendeley.com/datasets/5hpkf8v7bg/1

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, pp. 770–778, 2016.